

Handshake and State Machine Tutorial

Handshaking Protocols

Often, two modules need to share data but won't have any way of knowing when the data needs to be sent or is able to be received. To remedy this, handshaking protocols are used. The module sending or producing data and the module receiving or consuming data will have control signals to communicate readiness. A very simple handshake is used in this tutorial. The producer uses a "request" (**req**) signal to indicate data is ready to be sent and when the data has been sent. The consumer uses an "acknowledge" (**ack**) signal to indicate when it is ready to receive data, and when it has successfully captured the data. The details of this protocol are shown in the waveform in Figure 1 and the block diagram in Figure 2.

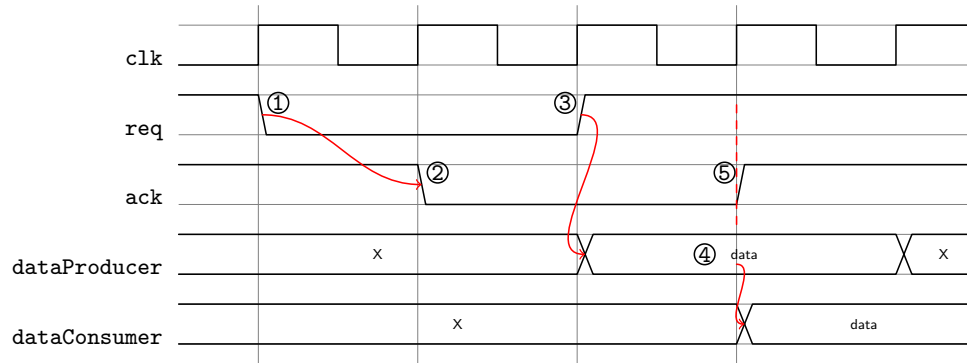


Figure 1: Handshaking timing diagram

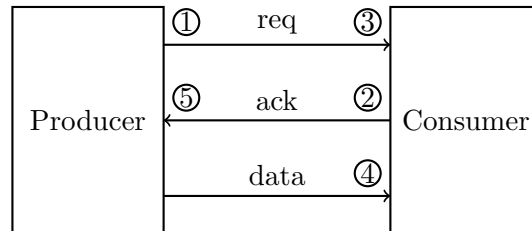


Figure 2: Block diagram of producer and consumer performing a handshake

The steps of this handshake are labelled on both Figure 1 and Figure 2 and are described as follows:

- ① The producer wants to send data and pulls **req** low
- ② The consumer sees the **req** signal is low and when it is ready to accept data it pulls **ack** low
- ③ The producer sees the **ack** signal is low, so it puts data on the data bus and pulls **req** high to indicate the data on the bus is what it wants to transmit
- ④ The consumer sees the **req** signal is high, so it captures the data on the bus
- ⑤ The consumer pulls **ack** high to indicate it is done capturing the data and is ready for more. The producer waits a cycle to ensure the data is valid long enough, however this could be skipped if there is no issue with hold time violation.

Both of these modules can be implemented as state machines. The state diagram for the producer is shown in Figure 3. To make testing easy, a “send” input is included in the producer so that the system this producer is instantiated in (in this case, just the test bench) can trigger the start of a transmission.

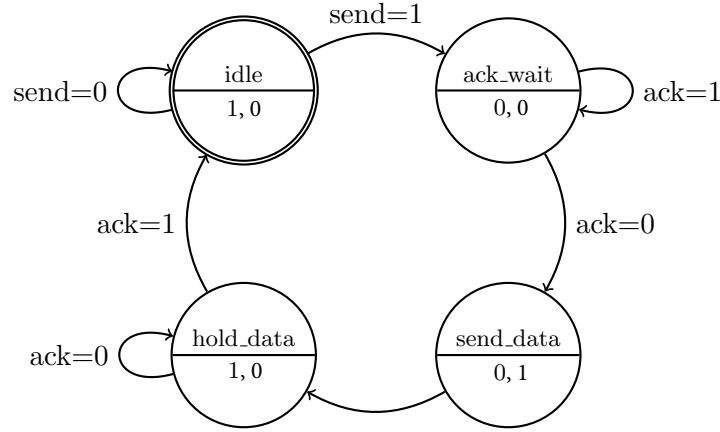


Figure 3: State diagram for producer. Output key: {req, d_en}

The state diagram for the consumer is shown in Figure 4. Mirroring the “send” input to the producer, a “recv” output is included to indicate that data was received.

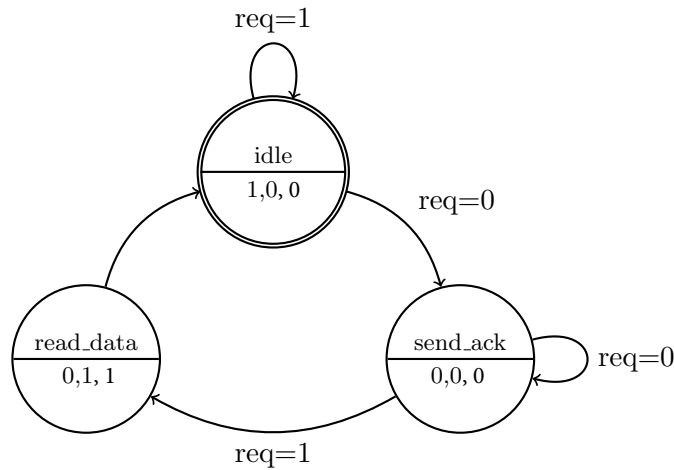


Figure 4: State diagram for consumer. Output key: {ack, recv, d_en}

State Machines

State machines can be implemented in several different ways. Two common coding styles are used in this tutorial. The producer state machine is written using “two” processes coding style and the consumer state machine is written using “three” processes style. Additionally, producer and consumer have additional “data path” process for managing the data.

1. Create a new Vivado project and import the given .vhd files. Look at the code and examine the

different state machine coding styles.

2. Elaborate the design and analyze both the producer and consumer.
3. Run a behavioral simulation using the provided test bench and analyze the behavior. Using the **Scope** pane to select a component and then the **Objects** pane, internal signals such as the state can be added to the waveform. Restart and run the simulation to see the waveforms for these added signals.
4. Run synthesis and open the synthesis report by clicking the **Reports** tab and double-clicking the item called **synth_1.synth_synthesis_report_0** (or something similar).
5. This report should say two FSMs were inferred, one for the producer and one for the consumer, and lists each state and its encoding. Pay attention to the encoding style.
6. Open the synthesized schematic and analyze the state FFs and output logic LUTs using the information learned from the state encodings found in the synthesis report. Compare the synthesis schematic to the elaborated schematic.
7. To force a particular coding style, right-click on **Synthesis** in the Flow Navigator, click **Synthesis Settings...**, and change the **-fsm-extraction** option as shown in Figure 5.

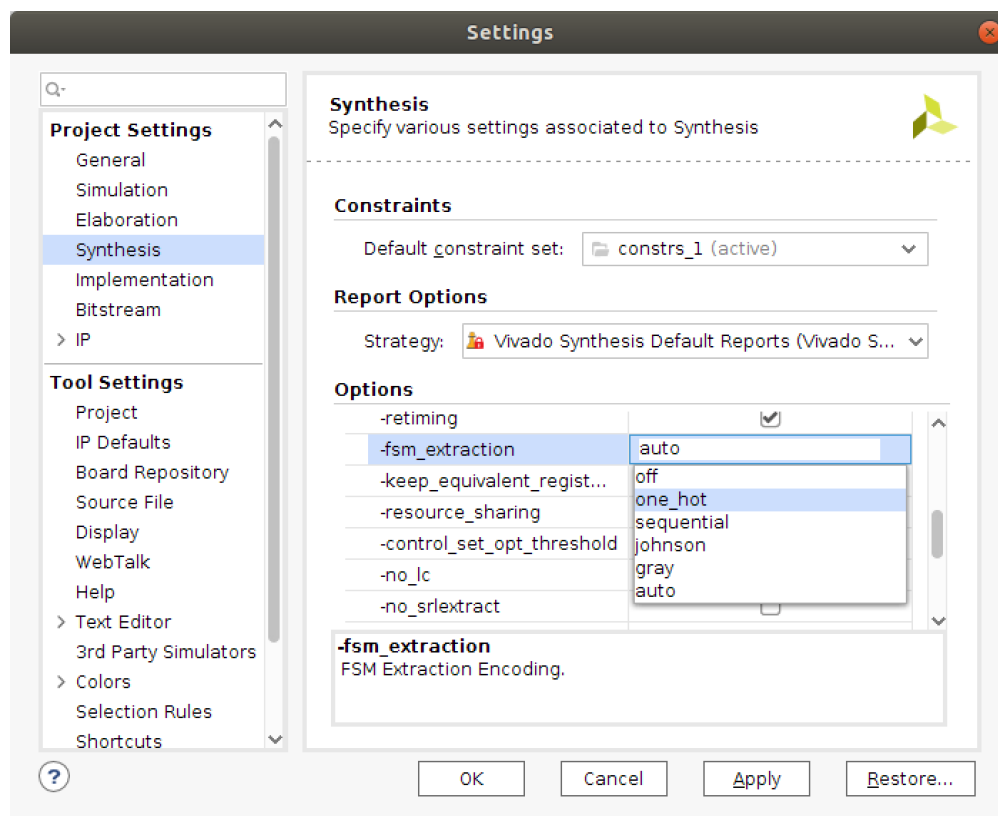


Figure 5: Synthesis settings

First change it to **one-hot**, click **OK**, and run synthesis again. Open the synthesis report and synthesis schematic and observe what changed. Record the number of flip flops used and explain

where each one comes from. For each of the producer and consumer, take note of how many FFs are used for state, for state machine outputs, and for the data signals.

8. Next, change the fsm-extraction option to **sequential**, resynthesize, and observe what happens. Record the new amount of flip flops and explain where each one comes from for each of the producer and consumer, as before. Compare this to the one-hot case and explain the difference.
9. Run a post-synthesis functional simulation. The original state signal with state names no longer exists, as it has been synthesized into a particular encoding. However, signals which have names similar to `FSM_onehot_state` or `FSM_sequential_state` are available. By adding these signals to the waveform and referencing the state encodings in the synthesis report, it is possible to determine what state the system is in. This is very useful when debugging post-synthesis or even post-implementation simulations.

Tutorial 2: Handshake and State Machine Tutorial

Student's Name: _____

Demo	Point Value	Points Earned	Date
Part 1: Producer and consumer state machine elaborated circuit diagrams. Coding style analysis	25		
Part 2: Behavioral simulation and analysis	25		
Part 3: Analysis of synthesis reports - number of FFs used for states and registers for two encoding styles	25		
Part 4: Post synthesis simulation and analysis for two encoding styles	25		

Students will need to demonstrate their understanding of all parts of this Tutorial by answering questions from Instructor/TAs.

Submit this grading sheet on myCourses under Assignemnts → Tut2.